Ders 1 Genişletilmiş Notu: Derse Giriş, Algoritma, Veri Yapısı ve Akış Diyagramı Kuralları

Ders 1 Genişletilmiş Notu: Derse Giriş, Algoritma, Veri Yapısı ve Akış Diyagramı Kuralları

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin Kapsamı ve Konu Akışı
- 2. Kodlamada Uyulacak Teknik Kurallar
- 3. Algoritmanın Kökeni
- 4. Algoritma ve Algoritma Tasarımı
- 5. Bir Algoritmanın Taşıması Gereken Özellikler
- 6. Genel Algoritmik Problemler
- 7. Veri ve Verinin Bellekte Saklanması
- 8. Veri Yapısı ve Neden Gerektiği
- 9. Algoritma Hazırlama Süreci
- 10. Sözde Kod
- 11. Akış Diyagramı Bileşenleri ve Kuralları
- 12. Algoritma Geliştirme ve İyileştirme
- 13. Asal Sayıları Bulma ve Eratosthenes Kalburu
- Örnek Sorular
- Ders Sonrası Notlar

1. Dersin Kapsamı ve Konu Akışı

Dr. Göksel Biricik, Veri Yapıları ve Algoritmalar (BLM2512) dersine bölümün temel derslerinden biri olarak başladı. Dönem online yürütüldüğü için dersler yüz yüze olduğu kadar etkileşimli geçemiyor. Hoca bu yüzden dersleri daha seri ve kompakt anlatacağını, kayıtları tekrar izleyerek konuların pekiştirilmesini önerdiğini söyledi. Dikkatin dağılmaması için her bölüm yaklaşık 40-45 dakika tutulacak, aralarda kısa molalar verilecek.

Derste kaynak olarak öncelikle sınıfta sunulan materyal kullanılır. Hoca slaytların yanında, özellikle akış diyagramlarında, slayt üzerine ya da boş sayfaya çizim yaparak ilerleyeceğini söyledi. Yararlanılan üç temel kitap şunlardır:

- Anany Levitin, *Introduction to the Design and Analysis of Algorithms* (2. ya da 3. baskı; aralarında küçük farklar var).
- Robert Sedgewick, *Algorithms* (Part 1 ve Part 2 olmak üzere iki ayrı kitap).
- Cormen ve diğerleri, *Introduction to Algorithms* (genellikle 3. baskı).

Hoca bu kitapları baştan sona okumayı değil, ilgili haftanın konusuna göz atmayı ve ek örnek problem çözmeyi önerdi.

Dönem boyunca işlenecek konuların sırası şöyle anlatıldı:

Hafta	Konu
1	Giriş: algoritmik problem, veri, veri yapısı, akış diyagramı tekrarı
2	Algoritma analizi, Büyük O (Big O) gösterimi ve karmaşıklık
3	Özyinelemeli (recursive) algoritma tasarımı ve analizi
4	Diziler, çok boyutlu diziler, bağlı liste (linked list)
5	Kuyruk (queue) ve yığın (stack)
6	Ağaç, ikili ağaç ve ikili arama ağacı (binary search tree)
7	Heap ve heap sort
8	Öncelikli kuyruk (priority queue)
11-12	Graf veri yapısı ve minimum yayılan ağaç (minimum spanning tree)
13	Karakter dizilerinde (string) etkin arama
14	Karşılaştırma yapmayan sıralama algoritmaları

Hoca heap sort konusunu, bir veri yapısının bir algoritmayı nasıl değiştirip daha etkin çalıştırabildiğini gösteren temel bir örnek olarak tanıttı. Öncelikli kuyruğu da heap'e dayanan, kuyruğun değiştirilmiş bir hâli olarak andı. Bu yapılarda her işlemin maliyetinin tartışılacağını söyledi. Son haftadaki sıralama algoritmalarının, ilk haftalarda görülecek karşılaştırmaya dayalı sıralamalardan farklı yaklaşımlar olacağını belirtti.

2. Kodlamada Uyulacak Teknik Kurallar

Hoca, dersteki bütün kodlama çalışmalarında yalnız ANSI standardındaki C dilinin kullanılacağını söyledi. C++ ve C# gibi diller başka dil sayılır. Bu kuralın arkasındaki teknik noktalar şunlardır:

- **Değişken tanımı bloğun başında yapılır.** C99 modunda değişkeni ilk kullanıldığı yerde tanımlamak mümkündür (örneğin `for (int i = 0; ...)`). ANSI C'de bu geçersizdir. Değişkenin tipi bloğun en tepesinde tanımlanır, değer kullanılacağı yerde atanır.
- **Derleyici:** Bölüm laboratuvarlarındaki Dev-C++ ve GCC 4.9.2 ile test edilmelidir. Hoca farklı derleyicilerde farklı modlar olabildiğini, bunun derleme sorununa yol açtığını söyledi.
- **Global değişken kullanılmaz.**
- **Büyük dizi ve matrisler için dinamik bellek ayrılır.** Bilgisayar Bilimlerine Giriş dersinde algoritma tasarımına odaklanıldığı için statik bellek kullanılıyordu. Artık dinamik bellek ayırma yöntemleri kullanılmalıdır.
- **Dosya yolu göreli (relative) verilir.** `C:\Users\user\Desktop\folder\input.txt` gibi mutlak yol yanlıştır. Programın bulunduğu klasöre göre `input.txt` yazmak doğrudur.
- **İstenen işlev kütüphane fonksiyonuyla değil, kendi kodunuzla yazılır.** Hocanın örneği: alt dizgi (substring) arama istenirse, bunu tek satırda yapan hazır fonksiyon çağrılmaz. Matematiksel bir fonksiyon istenirse `math.h` kullanılmaz.

Hoca ayrıca kodların çapraz tarandığını, değişken adlarını, döngü yapısını değiştirmenin ya da kontrolleri tersten yazmanın benzerliği gizlemediğini söyledi.

3. Algoritmanın Kökeni

"Algoritma" kelimesi Ebu Cafer Muhammed bin Musa El-Harezmi'nin adından gelir. El-Harezmi Harezm'de doğmuş, hayatı Bağdat'ta geçmiş bir gökbilimci ve matematikçidir. Hoca onun yaklaşık 780'de doğup 850'de Bağdat'ta öldüğünü söyledi. O dönemde bilgisayar bilimi yoktu; temel bilimler matematik ve gökbilimdi (astronomi).

Hoca, El-Harezmi'nin 825 civarında yazdığı ve Hint rakamlarıyla hesap yapmayı anlatan kitabın sonradan Latinceye *Algoritmi de numero Indorum* adıyla çevrildiğini anlattı. "Algoritmi", El-Harezmi adının Latince karşılığıdır ve "algoritma" kelimesinin tarihsel kökeni buraya dayanır.

4. Algoritma ve Algoritma Tasarımı

Algoritma tasarlamanın ilk şartı, elimizde çözüm gerektiren bir problemin olmasıdır. Bu çözüm de mutlaka işlemler yapılarak elde edilebilmelidir. İşlemsel çözüm gerektiren bir problemi çözmek için izlenecek yolun teorik olarak oluşturulmasına **algoritma geliştirmek** ya da **algoritma tasarlamak** denir.

Hocanın örnekleri:

- Elinizde bir sayı dizisi var ve küçükten büyüğe ya da büyükten küçüğe sıralamak istiyorsunuz. Bunu tanımlı işlemlerden oluşan adımlarla yapan yol bir algoritmadır.
- Navigasyon uygulamasında haritada iki nokta arasındaki en kısa ya da en hızlı yolu bulmak. Aynı problemi çözen farklı algoritmalar olabilir; amaca uygun olan seçilir.
- Bir zaman planı yapmak: "önce şunu yapacağım, sonra şunu çözeceğim". Bir ödevi yaparken kurduğunuz plan bile bir akış, bir algoritma tasarımıdır.
- Web'de arama yaptığınızda sonuçların hangi sırayla geleceği, ilgilerinin nasıl belirleneceği de algoritmik çözümlerle bulunur.

Tasarlanan algoritma Java, C, C++, C# gibi bir programlama diliyle ifade edildiğinde artık ona **program** denir.

4.1 Tanım

Algoritma, işlemsel olarak çözülebilen belirli bir problemi çözmek için arka arkaya uygulanacak **kesin** direktiflerden oluşan **sonlu** bir işlem kümesidir. Direktiflerin kesin olması ve sayılarının sonlu olması önemlidir. Algoritma, mantıklı ve doğru olan bütün girdiler için doğru sonucu üretmelidir. En önemli şartlardan biri de bu doğru çıktıyı sonlu bir zamanda üretmesidir. Sonsuz zamanda sonuç üretmek pratikte mümkün olmadığı için böyle bir şeye algoritma denmez. Bu şartların dışında kalan hiçbir şey algoritma değildir.

4.2 Günlük Hayattan Örnek: Diş Fırçalama

Hoca çok basit bir örnek olarak diş fırçalama akışını verdi:

1. Fırçayı al.
2. Üzerine diş macunu sür.
3. Dişlerine götür, ileri-geri ve yukarı-aşağı fırçala.
4. Suyla durula, ağzını çalkala.
5. Fırçayı suyla temizle.
6. Fırçayı ve macunu yerine koy.
7. Dur.

Hoca burada eksikler olduğunu da belirtti: örneğin macunun kapağının nasıl açılacağı söylenmemiş. Yine de bu, kesin direktiflerden oluşan sonlu sayıda adımlı bir eylem kümesidir. Doğru girdiler (diş fırçası, diş macunu, dişler, su) sağlanırsa çıktılar da üretilir: temizlenmiş dişler ve yerine konmuş fırça ile macun.

5. Bir Algoritmanın Taşıması Gereken Özellikler

Hoca, tanımda geçen "kesin, sonlu, doğru, girdi, çıktı" kavramlarını tek tek maddeledi:

- **Anlaşılırlık, belirsiz olmama (unambiguity):** Adımlar, girişler ve çıkışlar net olmalı; her biri tek bir anlam taşımalı. Muğlak ifadeler olmamalı.
- **Girdi (input):** Sıfır ya da daha fazla iyi tanımlanmış girdi. Bazı algoritmaların girdisi olmayabilir, ama genellikle en az bir girdi beklenir. Algoritmanın hangi girdilerle çalıştığı baştan bir varsayım olarak belirtilmelidir. Örneğin "bu algoritma yalnız tam sayılarla çalışır" denebilir.

- **Çıktı (output):** Çıktısı olmayan algoritma olmaz. Bir ya da daha fazla iyi tanımlanmış çıktı olmalıdır.
- **Sonluluk (finiteness):** Algoritma belirli bir adımdan sonra mutlaka sonlanmalıdır. Sonsuza kadar çalışan algoritma diye bir şey yoktur.
- **Yapılabilirlik (feasibility):** Elimizdeki mevcut imkânlarla gerçekleştirilebilen bir çözüm olmalıdır.
- **Bağımsızlık (independence):** Algoritma herhangi bir programlama dilinden, kodundan ya da kütüphanesinden bağımsız olmalıdır.

  Hocanın vurgusu: Bir algoritmayı ifade ederken hiçbir şekilde bir programlama dilinin gösterimi kullanılmamalıdır. Algoritmik çözüm, temel ve basit ifadelerle, ortamdan bağımsız olarak gösterilir.

6. Genel Algoritmik Problemler

6.1 Sıralama ve Algoritma Seçimi

Sıralama (sorting), bilgisayar bilimlerinde üzerinde en sık çalışılan algoritmik problemlerden biridir. Sıralama için iyi çalışan algoritmalar zaten vardır. Yine de daha iyisinin olup olmadığı araştırılır. Ayrıca hiçbir algoritma her ortamda ve her veri tipinde aynı performansla en iyi şekilde çalışmaz. Bazı ortamlarda bazı algoritmalar daha iyi çalışır ya da belli sebeplerle tercih edilir.

Peki neden A algoritmasını seçip B'yi seçmeyiz? Hoca iki ölçüt verdi:

- **Zaman (hız, çalışma zamanı):** Düz mantıkla, bir algoritma ne kadar karmaşıksa sonuç üretmesi o kadar uzun sürer. Aynı işi daha az karmaşıklıkla, dolayısıyla daha hızlı yapan yöntem tercih edilebilir.
- **Mekân (bellek):** Problemi çözerken ne kadar bellek harcandığı.

Hocanın örneği: Türkiye'deki bütün vatandaşların TC kimlik numaralarını sıralamak istiyorsunuz, yani yaklaşık 86 milyon elemanlı bir dizi var. Hızlı bir algoritma, hızlanmak için 86 milyonluk ek bir bellek alanı isteyebilir. Elinizde bu kadar yer yoksa o hızlı algoritmayı uygulamanın yolu yoktur. Bu, yapılabilirlik (feasibility) özelliğine bağlanır: Algoritma eldeki bellekle çözülemiyorsa sizin için uygun değildir. O durumda daha az bellek harcayan ama biraz daha yavaş çalışan bir algoritma seçilebilir.

> **Hocanın vurgusu:** Tek bir doğru yoktur. Üzerinde çalışılan probleme, ortamın kısıtlarına ve beklentilere göre aynı işi yapan pek çok algoritmadan biri seçilir ya da bu ölçütleri karşılayan yeni bir algoritma tasarlanır.

6.2 Arama

İkinci örnek, bir dizide belirli bir elemanı aramaktır (searching). Bir sayı dizisi ve bir x değeri verilir; x'in nerede olduğu bulunur. Baştan sona bakılabilir. Burada iki seçenek vardır: döngüyü sonuna kadar götürüp bulunan yeri söylemek ya da bulur bulmaz çıkmak. İkincisi tercih sebebidir.

> **Hocanın vurgusu:** Bulunduğu anda çıkmak, bir `for` döngüsünü `break` ile kırarak yapılmamalıdır. Algoritma tasarımında kesinlikle yapılmaması gerekenlerden biridir. Doğrusu, "ya sona gelene ya da bulana kadar" koşuluyla dönen bir `while` döngüsüdür.

Girdinin özellikleri iyi tanımlanmışsa bu özellikler kullanılabilir (exploit edilebilir). Girdi zaten sıralıysa, ikili arama (binary search) ile bir elemanın olup olmadığı ve varsa nerede olduğu çok daha az adımda, daha düşük karmaşıklıkla bulunur. Demek ki tasarımda girdinin yapısı ve özellikleri mutlaka göz önünde bulundurulmalıdır.

> **Ek bilgi (derste anlatılmadı):** İkili arama, sıralı dizinin ortasındaki elemana bakar. Aranan değer ortadakinden küçükse sol yarıda, büyükse sağ yarıda aramaya devam eder. Her adımda arama aralığı yarıya indiği için n elemanlı dizide en fazla yaklaşık $\log_2 n$ karşılaştırma gerekir. Doğrusal aramada bu sayı en kötü durumda n'dir.

6.3 Diğer Problem Türleri

- **Metin (string) işleme:** Karakter dizileriyle ilgili pek çok algoritmik problem vardır.
- **Graf problemleri:** Çok popüler ve çok çalışılan bir alan. Graf veri yapısıyla pek çok farklı problem güzel çözümlere kavuşur.
- **Kombinatoryal, geometrik ve nümerik problemler:** Hoca integral almanın bile bir iş olduğunu, bunun nasıl yapılacağının da bir algoritması olması gerektiğini söyledi.

7. Veri ve Verinin Bellekte Saklanması

Algoritmalar verileri girdi olarak alır ve işler. **Algoritmalar tarafından işlenen temel elemanlara veri denir.** Veri sayısal olabilir, metin (karakter) olabilir. Hoca, bugünkü bilgisayar mimarilerinde (temelde von Neumann mimarisi) her şeyin zaten sayısal olarak saklandığını vurguladı. Metin yalnızca bir temsildir; belli bir sayı örüntüsünün A harfi, B harfi olduğu, varsayılan dönüşümlerle kabul edilir.

- **Resim:** Görsel olarak anladığımız veri, aslında iki boyutlu bir matristir. İçindeki sayılar renklerin kodlanmış hâlidir. Farklı renk uzaylarında, farklı çözünürlük ve derinliklerde temsil edilebilir.
- **Ses:** Duyduğumuz frekanslardır. Sayısal ortamda algoritmalarla işlemek için yine arka planda sayısal olarak saklanır.

Girdi, çıktı ya da ara hesaplamalarda kullanılan bütün bilgiler algoritmaların işlediği verilerdir. Algoritmaların işleyeceği verilerin belirli bir düzende olması gerekir; yani verinin organizasyonu önemlidir.

7.1 Bitlerden Metne: “BABA” Örneği

Bilgisayarda veri yalnızca 0 ve 1 olarak, yani voltajın belli bir kademenin altında ya da üstünde olmasıyla tutulur. Bu hâliyle okunaklı değildir. 32 ya da 64 bitlik tam sayılar olarak yorumlanabilir. Daha kolay okumak için bitler 4’lü gruplanır ve her grup tek bir onaltılık (hexadecimal) basamakla yazılır. Örneğin `0100` grubu `4`, `0010` grubu `2` olur.

Hocanın slayttaki bit dizisi bu şekilde onaltılık olarak `42 41 42 41` diye okunur. Bu veri metin olarak yorumlanmak istenirse bir kod tablosu dönüşümüne ihtiyaç vardır:

Onaltılık	İkilik	Onluk	Karakter
`42`	`0100 0010`	66	`B`
`41`	`0100 0001`	65	`A`

Böylece `42 41 42 41` ekranda “BABA” olarak görünür. Siz ekranda “BABA” görürsünüz, ama bellekte 0 ve 1’lerden oluşan bir bit dizisi durur.

7.2 İşlemci ve Bellek

Hoca bir slaytta sağda belleği, solda işlemcinin üzerindeki yazmaçları (register) gösterdi. Yazmaçlar değişkenlerin değerlerini tutan özel alanlardır; slaytta 16 tane yazmaç vardı. Bir de program sayacı (program counter) bulunur. Program sayacı, işlemcinin çalıştıracağı bir sonraki komutun bellekteki adresini tutar.

Bellekte o işlemcinin komut setinde tanımlı makine kodları durur. Her komut “şu adresteki veriyi al, şu yazmaca yaz”, “bunu topla”, “bunu çıkar”, “bit işlemi uygula” gibi bir işe karşılık gelir. Program sayacı her komuttan sonra ilerler ve işlemci komut setindeki komutları adım adım çalıştırarak algoritmanın işlemesini sağlar. Bellekteki her şey yine 0 ve 1’dir; slaytta kolaylık olsun diye 16’lık tabanda gösterilmiştir. Farklı veri tiplerini, kabul ettiğimiz dönüşümlerle biz anlamlandırırız.

8. Veri Yapısı ve Neden Gerektiği

Dersin adı veri yapıları ve algoritmalar olduğu için veri yapısının da düzgün tanımlanması gerekir. **Algoritmaların erişme, değiştirme gibi işlemlerinde kullanılmak üzere bilginin saklandığı ve belirli özellikleri olan oluşumlara veri yapısı denir.**

Diziler bilinen bir yapıdır. Bağlı liste (linked list) gibi dinamik bağlantılı yapılar ise belirli problemlerin çözümünde büyük kolaylık sağlar. Bunların dışında yığın (stack), kuyruk (queue), hash tablosu, ağaç (tree), heap, öncelikli kuyruk (priority queue) ve daha ileri seviyede kümeler, sözlükler gibi yapılar vardır. Hepsi bilgiyi belirli bir formatta ve yapıda saklar, bilinen özellikleri ve işlemleri vardır.

Özetle, veri üzerinde çalışıp onu anlamlandırdığımızda ona bilgi deriz. Bu verilerin bellekte hangi düzende, hangi özellik ve işlemlere sahip yapılarda tutulacağını belirleyen oluşumlara veri yapısı denir.

8.1 Neden Gerekli: Seyrek Matris Örneği

Veri yapısı, veriyi etkin saklamak ve işlemek için gereklidir. Doğru yerde kullanılan çok basit bir veri yapısı bile algoritmik çözümü kolaylaştırabilir, hızlandırabilir.

Hocanın örneği: $n \times n$ boyutunda, $n = 1000$ olan bir matris oluşturmanız isteniyor. Yani $1000 \times 1000 = 1\,000\,000$ göz için bellek ayırmanız gerekiyor. Girdi hakkında hiçbir şey bilmiyorsanız bu bir milyon gözü ayırmak zorundasınız.

Ama girdinin bir özelliğini bildiğinizi düşünün: Matriste yalnız birinci sütunda, n'inci (son) sütunda ve köşegende ($(1,1), (2,2), \ldots, (n,n)$) değer var; kalan bütün değerler 0. Hoca bunu önce 4×4'lük bir matris çizerek gösterdi: köşegene 5, 3, 2, 1 gibi değerler, ilk ve son sütuna da değerler yazdı. 4×4'te bu anlamsız görünür, ama 1000×1000'de fark büyüktür.

Hesap şöyle kurulur:

- **Verilen:** $n = 1000$; dolu gözler ilk sütun, son sütun ve köşegen.
- **Dolu göz sayısı:** Her biri yaklaşık 1000 göz. Hoca kabaca "yaklaşık 3500" dedi.
- **Seyrek matris (sparse matrix) gösterimi:** Yalnız dolu gözler (satır, sütun, değer) üçlüsü olarak saklanır. Her dolu göz için 3 değer tutulur.
- **Hocanın hesabı:** $3500 \times 3 = 10\,500$ değer.
- **Karşılaştırma:** $1\,000\,000$ gözlük yer mi, $10\,500$'lük yer mi? Hoca "yaklaşık 100 kat daha az yer" dedi. Tabii ki ikincisi.

Doğru veri yapısını yerinde kullanmak hem zaman (çalışma hızı) hem mekân (bellekte kaplanan alan) olarak kazanç sağlar. Probleme ve ortama göre sizi kurtaran bir yardımcı olabilir.

> **Ders sonrası not:** Hoca dolu göz sayısı için "yaklaşık 3500" dedi. İlk sütun, son sütun ve köşegen 1000'er gözdür; $(1,1)$ hem ilk sütunda hem köşegende, (n,n) hem son sütunda hem köşegendedir. Doğru sayı $3 \cdot 1000 - 2 = 2998$'dir. Üçlü olarak $2998 \times 3 = 8994$ değer tutulur ve oran $1\,000\,000/8994 \approx 111$'dir. Hocanın "yaklaşık 100 kat daha az yer" sonucu geçerlidir.
>
> **Ek bilgi (derste anlatılmadı):** (satır, sütun, değer) üçlüleriyle saklamaya koordinat listesi (coordinate list, COO) biçimi denir. Bir matrisin elemanlarının çoğu 0 ise bu tür gösterimler bellekten büyük tasarruf sağlar. Karşılığında belirli bir gözün değerine erişmek, düz matristeki gibi doğrudan indisle değil, listede arayarak yapılır.

9. Algoritma Hazırlama Süreci

Hoca bir algoritmanın hazırlanmasını, "önce düşünmek, sonra bu düşünceyi hayata geçirmek" diye özetledi. Adımlar şunlardır:

1. **Tasarım (design):** Düşünce bir akışa, bir çözüme dökülür. Hoca bunun bir sistem geliştirmek değil, bir algoritma geliştirmek olduğunu vurguladı.
2. **İspat / doğrulama (validation):** Algoritmanın formal olarak doğru olduğu, yani doğru girdiler için doğru çıktılar ürettiği gösterilir.

3. **Analiz (analysis):** Girdilere göre ispatta olduğu gibi çıktılar üretip üretmediğine bakılır. Ayrıca, o ana kadar hiç yapılmamış bir şey yapılır: Algoritmanın karmaşıklığı ve sınırları ortaya konur. Basitçe, benzer algoritmalarla karşılaştırıldığında ne kadar hızlı çalıştığı ve ne kadar yer kapladığı incelenir. Ayrıntıları bir sonraki derste görülecek.
4. **Gerçekleştirme (implementation):** Algoritma çalışan bir koda dönüştürülür.
5. **Test:** Gerçek veriyle testler yapılır.

Algoritmaları ifade etmenin iki temel yolu vardır: sözde kod ve akış diyagramı.

10. Sözde Kod

Sözde kod (pseudo code), yarı programlama dili, yarı konuşma dili gibi bir gösterimdir. Hoca bu derste sözde kodu çok kullanmayacaklarını söyledi. Ancak internette algoritmalar, metin olarak yazması kolay olduğu için çoğunlukla sözde kodla verilir. Bu yüzden kuralları hızlıca anlatıldı.

10.1 Sözde Kodun Özellikleri

- Algoritmanın bir **adı** vardır. Bir fonksiyon gibi **girdileri** ve **çıktıları** yazılır. Girdinin ne olduğu tanımlanmalıdır; örneğin "n sayı" yerine "n tam sayı" ya da "kesirli sayı" demek daha doğrudur. Çıktının türü ve neyi ifade ettiği de yazılır.
- Gerekiyorsa parametreler, ön koşul ve son koşul yazılır. Geri dönüş değeri birden fazlaysa hepsi tanımlanır.
- Açılan her blok bir `end` ile kapatılır (`end if`, `end while`, `end for`). Bunu kullanmayan sözde kodlar da görülebilir, ama kullanmak okunurluğu artırır.
- **Veri elemanları tanımlanmaz.** İlk kullanıldıkları yerde otomatik olarak tanımlanmış sayılır; tipleri de atanan içerikle belirlenir. Ama bir **veri yapısı** kullanılıyorsa adıyla ve içeriğiyle mutlaka tanımlanmalıdır.
- Mümkünse adımlar numaralanır: 1, 1.1, 2, 3, 3.1, 3.1.1, 3.1.2, 3.2 gibi. Her iç blok bir alt numara alır.
- Değişkenler anlamlı adlandırılır. Tek harfli `X`, `Y`, `A`, `B` bir süre sonra okunmaz hâle gelir. "toplam", "ortalama" gibi genel adlar yerine neyin toplamı, neyin ortalaması olduğu belirtilmelidir.

10.2 Gösterim Kuralları

- **Atama:** Pascal'daki gibi `:=` ya da ok `←` ile gösterilir. Slayttaki örnekler: `max ← 1000` (max değişkenine 1000 atanır), `sum ← sum + newHeight`.
- **Eşitlik kontrolü:** Algoritmalar ortamdan bağımsız olduğu için tek `=` ile yapılır. C'deki `==` ya da JavaScript'teki `===` kullanılmaz.
- **Seçim:** `if (şart) ... end if`. `else` varsa: `if (şart) <doğruysa yapılacaklar> else <yanlışsa yapılacaklar> end if`. Bloklarda birden fazla eylem olabilir.
- **Döngüler:**
  - `while (şart) do ... end while`
  - `do ... while (şart)` ya da `repeat ... until (şart)`; bunlarda `end while` olmaz.
  - `for başlangıç to bitiş step artış do ... end for`
- **İndis:** Köşeli parantezle gösterilir: `arr[i]`, `ortalama[i+3]`.

10.3 Örnek: Dizinin Toplamı

Hocanın slayttaki örneği, verilen bir dizinin elemanlarının toplamını hesaplar:

```
Algoritma DiziToplami
  // Verilen dizinin elemanlarının toplamını hesaplar
  Girdi: n elemanlı tam sayı dizisi
  Çıktı: dizinin elemanlarının toplam değeri
  1. toplam ← 0
  2. for i ← 1 to n do
     2.1 toplam ← toplam + dizi[i]
  3. end for
```

```
  4. return toplam
end DiziToplami
```

Adım adım okunuşu:

- **1.** Toplama 0 atanır.
- **2.** i, 1'den n'e kadar döner. `step` verilmediği için artış 1'dir. İkişer ilerlemesi istenseydi `step 2` yazılırdı.
- **2.1.** Bu adım 2'nin içinde olduğu için alt numara alır. Toplama dizinin i'inci gözündeki değer eklenir. Bu adım n kez çalışır.
- **3.** Döngü kapanır.
- **4.** Elde edilen toplam, algoritmayı çağırana geri döndürülür.

11. Akış Diyagramı Bileşenleri ve Kuralları

Hoca, dersin asıl ağırlığının akış diyagramlarında (flowchart) olduğunu söyledi. Giriş derslerinde görülmüş olsa da zamanla unutulabildiği için bileşenler ve bağlantı kuralları tekrar edildi. Akış diyagramının bileşenleri:

- Başla ve bitir.
- Okuma (girdi).
- Bağlantılar.
- İfadeler ve işlemler (statement, process).
- Karar ifadeleri. Akışlarda genellikle `if` kullanılır; `switch-case` kullanılmaz.
- Döngüler: `while`, `for`, `do-while`.

11.1 Başla ve Bitir

Her algoritmanın bir başlangıç noktası olmalıdır. İçine "start", "başla" ya da "begin" yazılabilir; yeter ki başlangıç olduğu belli olsun. Genellikle oval (elips) ile gösterilir. **Tek bir başlangıç** olmalıdır.

Algoritmalar sonlu olduğu için akışın nerede bittiği de **tek bir bitiş** noktasıyla gösterilir. Hoca yanlış örneği şöyle anlattı: Sağdan "stop" çıkıyor, soldan "stop" çıkıyor, `if`'lerin her dalından bir yerlere "stop"a gidiliyor; salkım saçak, karman çorman bir görüntü. Bunun yerine bütün dallar tek bir "stop"ta toplanmalıdır.

11.2 Girdi Okuma

Algoritmada kullanılacak bütün girdilerin okunduğu mutlaka gösterilir. Okuma, delikli kart (punch card) sembolüyle gösterilir; kartın kenarı kesik çizilir. Hoca başka kaynaklarda klavyeden okuma için paralelkenar sembolünün görülebileceğini, ama bu derste ve bölümde kullanılmadığını söyledi.

- Okunan değişkenler kutu içinde virgülle ayrılarak net biçimde yazılır.
- Ortalama ya da toplam gibi değerler **okunmaz.** Okutulursa kullanıcıdan toplam değeri istenmiş gibi olur. Bunlar algoritmanın iç değişkenleridir; tanımlanmaz, ilk kullanıldıkları yerde otomatik olarak tanımlanmış sayılır.
- Dizi iki yolla okunabilir: Bir `for` döngüsü içinde indis indis okumak ya da kısa gösterimle (`A[n]` gibi) okumak.

  Hocanın vurgusu: Diziyi okumadan önce mutlaka boyutu okunmalıdır. Önce kaç eleman okunacağı (n), sonra dizinin o kadar elemanı okunur. Arada başka bir değişken (örneğin m) okunsa bile sıra bozulmaz.

11.3 Bağlantılar

Bileşenler oklarla bağlanır. Ok iki elemanı bağlar ve akışın yönünü gösterir. Oklar eğri büğrü değil, keskin köşeli çizilmelidir; böylesi daha okunaklıdır.

Akışlar büyüyebilir. Özellikle sınav gibi durumlarda unutulan bir parçayı sonradan bir yere eklemek gerekebilir. Bağlantıları basitleştirmek ve okunaklı kılmak için, özellikle iç içe döngü

ve kontrollerde, numaralı bağlantı noktaları kullanılır. Bunlarda akışın nereye girip nereden döndüğüne dikkat edilmelidir.

11.4 İşlem Kutusu

İfade ve işlemler dikdörtgenle gösterilir. İçinde bir aritmetik işlem, bir atama ya da başka bir işlem olabilir; ne yapıldığı net olmalıdır. Örneğin `A = A + 1`, A değişkeninin değerini 1 artırır.

> **Hocanın vurgusu:** Algoritmalar programlama dilinden bağımsızdır. Dile özel operatör ya da dile özel değişken yapısı kullanılmaz. `A = A + 1` yazılabilir, ama `A++` yazılamaz; çünkü bu C dilinin bir artım özelliğidir ve bütün dillerde desteklenmez.

11.5 Karar (`if`)

Başka üniversitelerden gelenler `if` için eşkenar dörtgen (baklava) sembolü görmüş olabilir: üstten girilir, iki koldan çıkılır. Hoca bunun daha eski bir gösterim olduğunu, içine yazmak zor olduğu için baklavayı ortadan açıp genişleterek **altıgen** kullanmayı tercih ettiklerini söyledi. Doğru ve yanlış çıkışları altıgenin sivri uçlarından çıkar.

Hocanın örneği: Koşul "A, B'ye eşit mi?" Doğruysa (true) `A = A + 2`, değilse (false) `A = A + 1`. İki kol sonra birleşir ve akış tek noktadan devam eder. Kollarda birden fazla işlem, hatta içte başka bir karar olabilir; onun da doğru ve yanlış kolları bir yerde birleşip dıştaki akışla buluşur.

Kurallar:

- Doğru ve yanlış kolları mutlaka **T** (true) ve **F** (false) diye etiketlenir.
- Okunaklılık için bütün `if`'lerde T kolları aynı tarafta olmalıdır. T'ler sağdaysa hepsi sağda olsun.
- **T kolu boş bırakılmaz.** En sık yapılan hatalardan biri, T kolunu boş geçip F koluna işlem yazmaktır: "A, B'ye eşitse hiçbir şey yapma, değilse şunu yap." Bu kesinlikle yanlıştır.
- F kolu isteğe bağlıdır. Yani `if-then-end if` olabilir, `if-then-else-end if` olabilir, ama yalnız `else`'i olan bir `if` olamaz.

Hoca bunu C ile de anlattı: `if (A == B) A = A + 2;` yazmak mümkündür; yalnız T kolu vardır. `else A = A + 1;` eklenirse F kolu da gelir.

Soru: T kolu boş olmamalıysa, "A, B'ye eşitse hiçbir şey yapma, değilse A'yı artır" nasıl çizilir?
Cevap: Koşul tersten yazılır: "A, B'ye eşit değil mi?" T kolunda `A = A + 1` olur. Eskiden F kolunda olan işlem böylece T koluna geçer.

11.6 `for` Döngüsü

`for`, iterasyon sayısı belli olan döngüler için kullanılır. Aslında bir `if`, bir artım ve artımdan sonra kontrol olarak da çizilebilir. Kolaylık için üç bölmeli (üç kompartımanlı) bir dikdörtgen kullanılır:

Bölme	İçerik
1	Döngü değişkeninin ilk değer ataması (← ya da = olabilir)
2	Nereye kadar döneceği, sonlanma değeri
3	Artım miktarı (step)

Örneğin j 1'den n'e birer birer artıyorsa okunuşu şöyledir: Döngüye girilir, $j = 1$ ile gövde çalışır, akış döngü kutusuna geri gelir. j bir artar, $j = 2$ olur. Böyle böyle n'inci adımda $j = n$ olur. Sonra j bir daha artar ve $n + 1$ olur. $n + 1 > n$ olduğu için döngüden çıkılır.

- Döngü gövdesi kutunun **sağ tarafına** çizilir. Gövde bitince ok döngü kutusuna geri saplanır.
- Döngü bitince akış kutunun **altından** devam eder.
- Gövdeyi soldan, alttan ya da çaprazdan çıkarmak doğru bir gösterim değildir.
- Döngüden `break` ile, kırarak çıkmak kesinlikle istenmez.

11.7 `while` Döngüsü

Dizide bir elemanı ararken sona kadar mı gitmeli, bulunca `break` ile mi çıkmalı? Hocanın cevabı: İkisi de değil, koşullu bir döngü kurulmalı. İterasyon sayısının bilinmediği ya da "belli bir koşul doğru olduğu sürece şu işi yap" denen durumlarda `while` kullanılır.

Hocanın örneği: "$j < n$ olduğu sürece" dönen bir `while`. Yukarıda bir yerde j'ye ilk değer verilmelidir, örneğin $j = 1$. Döngünün tek işlemi `A = A + 1` olsun. j değişiyor mu? Hayır. j hep 1'de kalır ve döngü sonsuza kadar döner.

> **Hocanın vurgusu:** `while` döngüsünün içinde bir yerde koşulu değiştiren bir işlem mutlaka olmalıdır. Tasarıma göre $j = j + 1$ ya da $j = j + A$ gibi bir şey yazılır; yeter ki koşul bir noktada bozulsun ve döngüden çıkılsın.

`while` döngüsünde de gövde sağ tarafa çizilir, işlemler yapıldıktan sonra ok yukarıda koşul kutusuna döner.

11.8 `do-while` Döngüsü

`do-while`'da önce gövdedeki işlemler çizilir, koşul onun altına konur. Koşul doğruysa ok yukarı, gövdenin başına saplanır; koşul sağlanmadığında akış aşağıya devam eder. Hoca tek bir işlem çizdi, ama gövdede pek çok işlem, döngü, dallanma ve koşul olabilir.

11.9 Örnek: Dizi Toplamının Akışı ve İzlenmesi

Hoca, sözde kodunu verdiği dizi toplamının akış diyagramını gösterdi: "Başla" ile başlanır, toplama ilk değer 0 atanır (ok ya da = ile, ikisi de kabul), sonra i için bir `for` döngüsü kurulur. Slaytta döngü 0'dan n'e yazılmıştı; hoca bunu 1 olarak düzeltti ve akışlarda genellikle 1'den n'e gitmenin daha doğru olduğunu söyledi. Gövdede `toplam = toplam + dizi[i]` vardır. Döngü bitince toplam ekrana yazılır ve akış biter.

Hoca bunu bir örnekle adım adım izledi. Bunun doğruluğu göstermenin (ispatın) bir yolu olduğunu söyledi.

- **Verilen:** $n = 3$; dizi: `dizi[1] = 3`, `dizi[2] = 5`, `dizi[3] = 8`.
- **Başlangıç:** toplam = 0.

Adım	i	İşlem	toplam
Başlangıç	-	-	0
1	1	$0 + 3$	3
2	2	$3 + 5$	8
3	3	$8 + 8$	16
Çıkış	4	$4 > 3$, döngüden çıkılır	16

- **Sonuç:** i'nin döngüden çıkıştaki değeri 4'tür. Ekrana 16 yazılır ve akış biter.

11.10 İç İçe Döngüler ve Numaralı Bağlantılar

Örneğin bir matrisi okumak ya da matris üzerinde işlem yapmak için iç içe iki `for` gerekir. İkinci `for` dıştaki döngünün sağındaki gövdeye konur, işlemler de iç döngünün gövdesine. İç döngü bitince akışın dış döngünün kutusuna dönmesi gerekir. Bunu uzun oklarla dolaştırmak bazen çok karışık olur.

Hoca burada numaralı bağlantı noktasını gösterdi: İç döngünün çıkışına içinde "1" yazan bir daire konur, dış döngü kutusunun dönüş noktasına da aynı "1" konur. Akış şöyle okunur: Dış `for`'a girilir, iç `for`'a girilir, işlemler yapılır; iç döngünün şartı bozulunca çıkılır ve "1'e git" denir. 1, dış döngünün dönüş noktasıdır. Dış döngü değişkeni bir artar ve iç döngüdeki işlemler yeniden, örneğin n kez, yapılır.

12. Algoritma Geliştirme ve İyileştirme

Bir problem, eldeki imkânlarla mümkün olan en iyi şekilde çözülmeye çalışılır. Bunun için önce **problemi doğru çözen basit bir temel algoritma** tasarlamak faydalıdır. Bu sayede küçük örneklerle analiz yapılabilir ve performans ölçümü için bir temel çizgi (baseline) elde edilir.

Sonra algoritma iyileştirilir: "Daha iyi ne yapabilirim, bunu nasıl tasarlayabilirim?" diye düşünülür. Veri yapıları ve algoritmalar her zaman birlikte düşünülmelidir. Bazen algoritma çok basittir ama çok karmaşık bir veri yapısı kullanır; bazen de algoritma çok karmaşıktır ama yalnız dizilerle, yani basit bir veri yapısıyla çalışır.

Neden daha iyi algoritma geliştirmek isteriz? Hocanın örneği: Algoritmanızı hiç uğraşmadan en basit şekilde yazdınız ve daha hızlı çalışmasına ihtiyacınız var. Bilgisayarınızın ve işlemcinizin hızı bellidir. Para harcayıp 10 ya da 100 kat güçlü bir bilgisayar alabilirsiniz. Ya da biraz düşünüp algoritmanızı 10 kat hızlı çalışacak şekilde tasarlayabilir, aynı bilgisayarla aynı çözümü 10 kat hızlı elde edebilirsiniz.

Hangi algoritmanın seçileceği her zaman kolay cevaplanan bir soru değildir; biraz deneyim işidir. Temelde zaman ve mekân karmaşıklığı önemlidir ve bazen biri diğerine tercih edilir:

- **Gömülü sistem:** Bellek çok kısıtlıdır. "Yavaş ve uzun çalışsın, ama bellek harcamasın" denebilir.
- **Borsa işlemleri yapan sistem:** "Bellek önemli değil, gerekirse gigabaytlarca bellek takarız; yeter ki çok hızlı çalışsın" denebilir.

13. Asal Sayıları Bulma ve Eratosthenes Kalburu

13.1 Problem ve Basit Çözüm

Hoca, 1'den n'e kadar bütün asal sayıların nasıl bulunacağını sınıfa sordu (ayrıntısı Örnek Sorular bölümünde). Akla gelen ilk yol, her sayıyı kendisine kadar olan bütün sayılara bölmektir. Bu, iç içe iki `for` demektir: Dışta 1'den n'e, içte yine 1'den n'e dönülür. n sayı için n'er işlem, yani $n \cdot n = n^2$ işlem yapılır. Sonra bir döngüyle bölünemeyen sayılar yazdırılır. Hoca iç içe iki `for`'un çarpımlı bir maliyet demek olduğunu söyledi.

İlk iyileştirme: Bölenleri sayının yarısına kadar denemek yeterli olabilir. Bu durumda işlem sayısı yaklaşık $n^2/2$ olur. Hoca bir ipucu daha verdi: $\sqrt{n}$'e kadar denemek yeterlidir, çünkü $\sqrt{n} \cdot \sqrt{n} = n$'dir.

13.2 Eratosthenes Kalburu

Daha iyisi yapılabilir mi? Bunun bilinen bir yöntemi Eratosthenes Kalburu'dur (Sieve of Eratosthenes). Hoca slaytta Eratosthenes'in antik Yunan'dan kalma bir tasvirini de gösterdi.

Yöntemin fikri: İlk görülen sayı asaldır ve silinmez; onun katları ise hiçbir zaman asal olamaz ve silinir. Hoca 1'den 20'ye kadar şöyle uyguladı:

- **2:** Kalır. Katları silinir: 4, 6, 8, 10, 12, 14, 16, 18, 20.
- **3:** Sıradaki silinmemiş sayı, kalır. Katları silinir: 6 zaten silinmişti; 9 ve 15 silinir. Silinmiş sayılara bir daha bakmaya gerek yoktur.
- **5:** Kalır. Katları 10 ve 15 zaten silinmiştir.
- **7:** Kalır. Katı 14 zaten silinmiştir; 21 ve sonrası 20'den büyüktür.
- Hoca $\sqrt{20}$'ye kadar gitmenin yeterli olduğunu söyledi; ondan sonraki sayıların hepsi bir şekilde silinmiş durumdadır.
- **Sonuç:** 2, 3, 5, 7, 11, 13, 17, 19.

  Ders sonrası not: $\sqrt{20} \approx 4{,}47$ olduğu için kalbur aslında yalnız 2 ve 3 için çalıştırılmalıdır. Derste 5 ve 7'ye de bakıldı; bu adımlar sonucu değiştirmez ama gereksizdir. Nitekim 5 ve 7'nin 20'ye kadarki katları zaten silinmiş çıktı.

Slayttaki sözde kod şöyledir:

```
Input: Integer n >= 2
Output: n'den küçük eşit asal sayılar
```

```
for p <- 2 to n do A[p] <- p

for p <- 2 to floor(sqrt(n)) do
    if A[p] != 0              // p önceden listeden silinmediyse
        j <- p * p
        while j <= n do
            A[j] <- 0         // asal değil olarak işaretle
            j <- j + p
```

Hoca önce `A[p] <- p` ile 2'den n'e kadar sayıları tutan diziyi oluşturduğunu söyledi; slaytta matris gibi görünse de bu $1 \times n$'lik bir dizidir. Sonra 2'den $\sqrt{n}$'e kadar, daha önce silinmemiş her sayının kalan bütün katlarını bir döngü içinde siler. Silmek, gözüne 0 yazmak demektir.

Hoca bu algoritmanın akışının iki farklı şekilde ifade edilebileceğini, bunların bir sonraki derste inceleneceğini söyledi.

> **Ek bilgi (derste anlatılmadı):** Silmeye p^2'den başlanmasının sebebi, p'nin p^2'den küçük katlarının ($2p, 3p, \ldots$) daha küçük bir asalın katı olarak zaten silinmiş olmasıdır. Eratosthenes Kalburu'nun çalışma süresi $O(n \log \log n)$'dir; bu, her sayıyı $\sqrt{n}$'e kadar bölmeye dayanan yöntemden çok daha hızlıdır.

Örnek Sorular

1. (derste sözlü, 01:33:16) 1'den n'e kadar olan bütün asal sayıları nasıl bulabiliriz?

Derste verilen cevap:

- Sınıftan gelen ilk fikir bir `for` döngüsüyle sayıları 2, 3, 5, 7'ye bölerek kontrol etmekti. Hoca 7'den sonra 11'e, 17'ye de bakmak gerekeceğini hatırlattı.
- Basit çözüm: Her sayıyı kendisine kadar olan bütün sayılara böl. İç içe iki `for` gerekir, yaklaşık n^2 işlem. Sonra bölünemeyenleri yazdır.
- İyileştirme: Yarısına kadar bölmek yaklaşık $n^2/2$ işlem; $\sqrt{n}$'e kadar bölmek yeterlidir.
- Daha iyisi: Eratosthenes Kalburu (bkz. Bölüm 13.2). 20 için sonuç 2, 3, 5, 7, 11, 13, 17, 19.

2. (derste sözlü, 01:20:35) "A, B'ye eşitse hiçbir şey yapma, değilse A'yı 1 artır" işlemi, T kolu boş kalmadan akış diyagramında nasıl çizilir?

Derste verilen cevap: Koşul tersine çevrilir. Karar kutusuna "A ≠ B" yazılır, T koluna `A = A + 1` konur, F kolu boş kalabilir. İki kol aynı noktada birleşir.

3. (derste sözlü, 01:23:09) Dizide bir eleman aranırken sona kadar mı gidilmeli, yoksa bulunca `break` ile mi çıkılmalı?

Derste verilen cevap: İkisi de değil. "Sona gelene ya da bulana kadar" koşuluyla dönen bir `while` döngüsü kurulmalıdır. Döngünün içinde koşulu değiştiren bir işlem bulunmalıdır.

> **Çözüm (derste verilmedi):** Bu kurala uyan ANSI C kodu aşağıdadır. Dizi dinamik ayrılır, `break` kullanılmaz; döngü indis sona ulaşınca ya da eleman bulununca biter. Kod `gcc -Wall -ansi -pedantic` ile uyarısız derlendi; `5 / 4 9 2 7 1 / 7` girdisiyle "7, 3. indiste bulundu" çıktısını verdi (indis 0'dan sayılır).

```c
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int n, x, i, bulundu;
    int *dizi;

    if (scanf("%d", &n) != 1 || n < 1) {
        return 1;
    }
    dizi = (int *) malloc(n * sizeof(int));
```

```
    if (dizi == NULL) {
        return 1;
    }
    for (i = 0; i < n; i++) {
        scanf("%d", &dizi[i]);
    }
    scanf("%d", &x);
    i = 0;
    bulundu = 0;
    while (i < n && bulundu == 0) {
        if (dizi[i] == x) {
            bulundu = 1;
        } else {
            i = i + 1;
        }
    }
    if (bulundu == 1) {
        printf("%d, %d. indiste bulundu\n", x, i);
    } else {
        printf("%d dizide yok\n", x);
    }
    free(dizi);
    return 0;
}
```

4. (derste sözlü, 01:37:56) Eratosthenes Kalburu'nun akışı nasıldır? Hoca bunu bir sonraki derse kadar düşünülmesi ve kendi başına çizilmesi için bıraktı.

> **Çözüm (derste verilmedi):** Akış sözel olarak şöyle kurulur: Başla → n'i oku → `for` p: 2'den n'e, 1'er (gövde: $A[p] = p$) → `for` p: 2'den $\lfloor\sqrt{n}\rfloor$'e, 1'er; gövdede karar "$A[p] \neq 0$?" → T kolunda $j = p \cdot p$, ardından `while` "$j \leq n$?" (gövde: $A[j] = 0$, $j = j + p$) → döngüler bitince `for` p: 2'den n'e; "$A[p] \neq 0$?" T ise $A[p]$'yi yaz → Bitir. Aşağıdaki ANSI C kodu aynı adımları izler; `break` ve global değişken yoktur, dizi dinamik ayrılır. Kod `gcc -Wall -ansi -pedantic` ile uyarısız derlendi; $n = 20$ için `2 3 5 7 11 13 17 19` yazdı.

```
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int n, p, j;
    int *A;

    printf("n (n >= 2): ");
    if (scanf("%d", &n) != 1 || n < 2) {
        printf("Gecersiz girdi\n");
        return 1;
    }
    A = (int *) malloc((n + 1) * sizeof(int));
    if (A == NULL) {
        printf("Bellek ayrilamadi\n");
        return 1;
    }
    for (p = 2; p <= n; p++) {
        A[p] = p;
    }
    for (p = 2; p * p <= n; p++) {
        if (A[p] != 0) {
            j = p * p;
            while (j <= n) {
```

```
                A[j] = 0;
                j = j + p;
            }
        }
    }
    for (p = 2; p <= n; p++) {
        if (A[p] != 0) {
            printf("%d ", A[p]);
        }
    }
    printf("\n");
    free(A);
    return 0;
}
```

Not: Koddaki `p * p <= n` koşulu, `floor(sqrt(n))`'e kadar dönmekle aynıdır ve `math.h` gerektirmez.

Ders Sonrası Notlar

> **Ders sonrası not:** Hoca 4 bitlik gruplamaya "binary coded decimal" dedi. Yaptığı şey onaltılık (hexadecimal) gösterimdir. BCD, her onluk basamağı ayrı 4 bitle kodlar ve yalnız 0-9 değerlerini kullanır.
>
> **Ders sonrası not:** Hoca *Algoritmi de numero Indorum* çevirisini "Algebra" adlı kitaba bağladı ve çevirinin 1300'lerde yapıldığını söyledi. Bu Latince çeviri, büyük olasılıkla El-Harezmi'nin Hint rakamlarıyla hesap üzerine yazdığı aritmetik kitabına aittir ve 12. yüzyılda yapılmıştır. Cebir kitabı ayrı bir eserdir: *el-Kitâb el-Muhtasar fî Hisâb el-Cebr ve'l-Mukâbele*.
>
> **Ders sonrası not:** Altyazıda "fitness" diye geçen özelliğin doğrusu sonluluk (finiteness), "anambigüte" diye geçenin doğrusu belirsiz olmama (unambiguity)'dır.
>
> **Ders sonrası not:** Hoca yazmaçları "R0'dan RFE'ye kadar 16 tane" diye okudu. 16 yazmaç onaltılık olarak R0'dan RF'ye kadar numaralanır.
>
> **Ders sonrası not:** $\sqrt{n}$ kuralının gerekçesi: n bileşik sayıysa $n = a \cdot b$ yazılır ve a ile b'den en az biri $\sqrt{n}$'den küçük ya da eşittir. Yani $\sqrt{n}$'e kadar böleni olmayan sayı asaldır. Derste söylenen "ondan sonrakilerin hepsi öncekilerin çarpımı olmak durumunda" ifadesi bu kuralın tam karşılığı değildir.

Dersin Toparlaması

Bu derste algoritmanın tanımı, kökeni ve taşıması gereken özellikler (belirsiz olmama, iyi tanımlı girdi ve çıktı, sonluluk, yapılabilirlik, dilden bağımsızlık) görüldü. Algoritma seçiminde zaman ve bellek arasındaki dengenin probleme ve ortama göre değiştiği, tek bir "en iyi" algoritma olmadığı vurgulandı. Veri, bellekte 0 ve 1'lerden oluşan ve dönüşümlerle anlamlandırılan bir temsil olarak ele alındı. Veri yapısının neden gerektiği, seyrek matris örneğiyle yaklaşık 100 kat bellek kazancı üzerinden gösterildi.

Algoritmalar sözde kod ve akış diyagramıyla ifade edilir. Akış diyagramında tek başlangıç ve tek bitiş, dile özgü operatör kullanmama, T kolunu boş bırakmama, `break` kullanmama ve `while` içinde koşulu değiştirme kuralları öne çıktı. Ders, asal sayı bulma problemiyle algoritma iyileştirmeye geçti: n^2'lik basit yöntemden $\sqrt{n}$ sınırına ve Eratosthenes Kalburu'na varıldı. Kalburun akış diyagramının iki farklı biçimi bir sonraki derste incelenecek. Sonraki derste ayrıca algoritma analizine, Büyük O gösterimine ve karmaşıklık kavramına geçilecek.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.